

Optimus-Cost-Agent

Low-Level Design (LLD) Specification

Gateway-Centric Cost Governance, Tooling, Observability & Runtime Control

Version 2.38

Architected by: Vibhanshu Agarwal

0. Optimus AI Gateway Architecture

The Optimus AI Gateway is Phase 1 mandatory infrastructure, not future scope. All traffic from the local agent — model completions and every tool call — flows through the gateway from day one. There is no supported configuration in which the local agent calls a provider directly.

Asking every developer to supply a Tavily key, an OpenAI key, and any other provider credential creates credential sprawl and undermines the cost-control story. The correct model is single-credential access: the developer authenticates once with Optimus, and the gateway holds all provider credentials server-side in a Vault. This mirrors the model used by Cursor, JetBrains AI, and similar developer tools, and it is the only architecture Optimus ships with.

A. Recommended Architecture

The control plane is entirely on the gateway side. The local IDE plugin or local Optimus agent only knows two things:

```
OPTIMUS_GATEWAY_URL=https://gateway.optimus.ai
OPTIMUS_API_KEY=opt_live_xxx

# Or, for IDE integrations:
# Sign in with Optimus (OAuth / device flow)
```

The gateway then maps that credential to the org / user / project context and resolves budgets, allowed tools, provider routes, and backend secrets internally.

B. Gateway Component Flow

The following describes the request path from IDE plugin to downstream providers:

```
IDE Plugin / Local Optimus Agent
  | 1 Optimus key or OAuth token
  v
Optimus AI Gateway
├── Auth + Project Policy (validates credential, resolves org/project)
├── Model + Tool Router   (selects provider, resolves tool endpoint)
├── Cost Ledger / Budget Engine (enforces spend caps, attribution)
├── Secret Vault          (holds provider credentials, never exposed)
├── ────────────────────▶ OpenAI / OpenRouter / Azure OpenAI
├── ────────────────────▶ POST /v1/tools/web/search (Tavily, proxied)
├── ────────────────────▶ POST /v1/tools/web/extract (Tavily, proxied)
```

C. Gateway Responsibilities

- **Single developer authentication** — one credential, one session.
- **Model routing** across OpenAI / OpenRouter / Azure and other LLM providers.
- **Tool brokering** for Tavily, OSV, package registries, and MCP tools.
- Centralised prepaid balance or subscription billing.
- **Cost attribution** by org_id, user_id, project_id, run_id.
- **Policy enforcement:** Plan mode, Agent mode, tool call caps, allowed domains.
- **Secret isolation:** provider keys live in Vault, never on developer machines.
- Caching and de-duplication to amplify the cost-savings story.

D. Gateway-Facing API Shape

The gateway exposes OpenAI-compatible model endpoints where possible, plus typed tool endpoints:

```
# Model completions — Responses API shape (uses "input" field)
POST /v1/responses
Authorization: Bearer opt_live_xxx
{ "model": "optimus-default", "input": "..." }

# Model completions — Chat Completions shape (uses "messages" array)
POST /v1/chat/completions
Authorization: Bearer opt_live_xxx
{ "model": "optimus-default", "messages": [...] }

# Do NOT mix shapes: "messages" at /v1/responses or "input" at
# /v1/chat/completions will be rejected by the gateway schema validator.

# Internal tool endpoints (called by local agent via gateway)
POST /v1/tools/web/search
POST /v1/tools/web/extract
POST /v1/tools/package/lookup
POST /v1/tools/security/advisory
```

E. Developer-Facing vs. Server-Side Configuration Boundary

This boundary is strict. The table below shows which configuration belongs where:

Developer machine (local agent LLD)	Gateway / Vault (server-side only)
OPTIMUS_GATEWAY_URL, OPTIMUS_API_KEY (or OAuth session); production_mode flag; enterprise origins via signed tenant profile only	tavily_api_key, openai_api_key, openrouter_api_key, etc.

0A. Local vs. Gateway Configuration & Provider-Cost Mapping

This subsection extends Section 0 and the Developer-Facing vs. Server-Side Configuration Boundary (§0E). The architectural rationale is in the HLD (§5A); the coverage / observability target is owned by the Test Strategy (§8A).

Phase 1 Runtime Configuration Rule

Phase 1 local runtime configuration allows only `OPTIMUS_GATEWAY_URL` and `OPTIMUS_API_KEY` (or an equivalent OAuth / device-flow session). Local `TAVILY_API_KEY`, `OPENAI_API_KEY`, `OPENROUTER_API_KEY`, and `LANGSMITH_API_KEY` (and any other provider or observability key) are rejected for Phase 1 runtime use. These keys are owned by the Gateway / deployment layer and held server-side in Vault.

Provider-Cost Mapping

`OPTIMUS_API_KEY` is not a physical upstream key; it is an internal account / wallet key. The Gateway maps it to a tenant / user / project / budget wallet and converts heterogeneous upstream costs into one normalized Optimus ledger. Vendors such as Tavily, LangSmith, OpenAI, and OpenRouter each expect their own credential and bill the Optimus org account; they never consume Optimus credits directly. Provider-native units are not hardcoded in the agent — they are whatever the provider reports, normalized by the Gateway adapter.

```
LLM call:
  upstream unit: input / output tokens
  gateway records:
    provider          = openai | openrouter | anthropic | glm | ...
    native_unit       = tokens
    billing_units     = input_tokens + output_tokens
    cost_usd          = price_snapshot(model) applied to usage
    optimus_credits_debited = normalized internal charge

Tavily search:
  upstream unit: Tavily credits
  gateway records:
    provider          = tavily
    native_unit       = tavily_credits
    billing_units     = provider-reported Tavily credits,
                      as normalized by the Gateway adapter
    cost_usd          = gateway-computed from the active price snapshot
    optimus_credits_debited = normalized internal charge

LangSmith trace:
  upstream unit depends on the commercial model:
    • usage-based      → trace / span / event counts
    • seat/subscription → allocated or amortized observability cost
  gateway records native trace usage OR allocated observability cost
  accordingly; cost_usd is computed or amortized into the same ledger.
```

Gateway Tool & Observability Endpoints

The local agent never calls Tavily or LangSmith directly. It calls Gateway-owned adapters; the Gateway injects the server-side vendor key, records usage, normalizes cost, and returns a unified envelope (gateway_request_id, cost_usd, billing_units, cache_hit, citations where applicable).

```
# Model completions
POST /v1/responses          # Responses API shape ('input')
POST /v1/chat/completions   # Chat Completions shape ('messages')

# Gateway-owned typed tools
POST /v1/tools/web/search    Authorization: Bearer OPTIMUS_API_KEY
POST /v1/tools/web/extract
POST /v1/tools/package/lookup
POST /v1/tools/security/advisory

# Gateway-managed observability
POST /v1/observability/traces
```

Canonical Phase 1 LangSmith wiring (single primary path). The local agent emits structured trace events to the Gateway at /v1/observability/traces; the Gateway exports to LangSmith using a server-side service key. Direct Gateway-native spans to LangSmith are retained only as an internal extension, not an equal option. If LangSmith is proxied, LANGSMITH_ENDPOINT is forced to the Optimus Gateway and the real LangSmith key exists server-side only.

1. ACP Protocol Framing & JSON-RPC Contract Specification

To prevent streaming fragmentation errors, the ACP interface enforces a strict HTTP-style byte framing header layout. Malformed payloads trigger immediate JSON-RPC standard error blocks.

A. Header, Buffer Size Limits & Serialization Standards

Every frame must prepend an explicit Content-Length prefix sequence. To prevent Denial of Service (DoS) attacks, the transport plane enforces a strict Maximum Header Size of 8 KB and a Maximum Body Size of 10 MB. Responses echo a matching structural correlation integer via the id field.

```
[INBOUND SYSTEM STREAM TRANSACTION]
Content-Length: 114
```

```
{
  "jsonrpc": "2.0",
  "id": 42,
  "method": "agent/prompt",
  "params": {
    "prompt": "Refactor...",
    "targetFile": "main.py"
  }
}
```

```
PARSE_ERROR = {
  "code": -32700,
  "message": "Parse error: Invalid JSON payload received."
}
INVALID_REQUEST = {
  "code": -32600,
  "message": "Invalid Request: Schema mapping parameters missing."
}
INTERNAL_ERROR = {
  "code": -32603,
  "message": "Internal error: Underlying task execution failure."
}
ID_COLLISION_ERROR = {
  "code": -32001,
  "message": "Server error: Request ID collision detected."
}
```

2. Cross-Platform Stream Transport Layer

```
import asyncio
import sys
import json
import re
from typing import Optional, Tuple
from json import JSONDecodeError
from concurrent.futures import ThreadPoolExecutor

class CrossPlatformStreamReader:
    def __init__(self, max_header_bytes: int = 8192,
                 max_body_bytes: int = 10485760):
        self.executor = ThreadPoolExecutor(max_workers=1)
        self.max_header = max_header_bytes
        self.max_body = max_body_bytes

    def sync_blocking_read(self) -> Tuple[Optional[bytes], bool]:
        header = b""
        while b"\r\n\r\n" not in header:
            if len(header) >= self.max_header:
                return None, True
            char = sys.stdin.buffer.read(1)
            if not char:
                return b"", False
            header += char
        match = re.search(b"(?i)Content-Length\s*:\s*(\d+)", header)
        if not match or int(match.group(1)) > self.max_body:
            return None, True
        content_length = int(match.group(1))
        return sys.stdin.buffer.read(content_length), False

    async def read_frame(self) -> Tuple[Optional[dict], Optional[dict]]:
        loop = asyncio.get_running_loop()
        raw_bytes, bad_frame = await loop.run_in_executor(
            self.executor, self.sync_blocking_read
        )
        if bad_frame:
            return None, PARSE_ERROR
        if not raw_bytes:
            return None, None
        try:
            return json.loads(raw_bytes.decode("utf-8")), None
        except JSONDecodeError:
            return None, PARSE_ERROR
```

3. Queued Task Lifecycle, Backpressure Controls & Runtime Pooling

```
import httpx
import uuid
import redis.asyncio as aioredis

class ManagedTaskManager:
    def __init__(self, max_concurrent_prompts: int = 3,
                 execution_timeout_secs: int = 120):
        self.semaphore = asyncio.Semaphore(max_concurrent_prompts)
        self.timeout = execution_timeout_secs
        self.all_tracked_tasks = {}
        self.http_pool = None
        self.redis_pool = None

    async def initialize_runtime_pools(self, redis_url: str):
        if self.all_tracked_tasks:
            raise RuntimeError(
                "Cannot reinitialize pools while tasks are running."
            )
        if self.http_pool or self.redis_pool:
            if self.http_pool:
                await self.http_pool.aclose()
            if self.redis_pool:
                await self.redis_pool.disconnect()
        self.http_pool = httpx.AsyncClient(
            limits=httpx.Limits(
                max_connections=10, max_keepalive_connections=5
            ),
            timeout=httpx.Timeout(30.0),
        )
        self.redis_pool = aioredis.ConnectionPool.from_url(
            redis_url, max_connections=20, decode_responses=True
        )
```

```
    async def enqueue_and_execute(self, request_id: Optional[int],
                                  pipeline_coro_factory) -> dict:
        active_id = request_id if request_id is not None \
            else int(uuid.uuid4()).int & 0x7FFFFFFF
        if not self.http_pool or not self.redis_pool:
            return {
                "status": "ERROR",
                "error": {
                    "code": -32003,
                    "message": "Server error: Connection pools uninitialized.",
                },
            }
        if active_id in self.all_tracked_tasks:
            return {
                "status": "ERROR",
                "error": {
                    "code": -32001,
                    "message": "Server error: Request ID collision detected.",
                },
            }

    async def _orchestrated_run():
        async with self.semaphore:
            coro = pipeline_coro_factory(self.http_pool, self.redis_pool)
            return await asyncio.wait_for(coro, timeout=self.timeout)

    loop = asyncio.get_running_loop()
    wrapper_task = loop.create_task(_orchestrated_run())
    self.all_tracked_tasks[active_id] = wrapper_task
    try:
        result = await wrapper_task
        return {"status": "SUCCESS", "data": result}
    except asyncio.CancelledError:
        return {
            "status": "ERROR",
            "error": {"code": -32603, "message": "Task cancelled during shutdown."},
        }
    except asyncio.TimeoutError:
```

```
        "error": {"code": -32603, "message": "Execution timeout."},
    }
    finally:
        self.all_tracked_tasks.pop(active_id, None)

    async def graceful_shutdown(self):
        for task in list(self.all_tracked_tasks.values()):
            task.cancel()
        if self.all_tracked_tasks:
            await asyncio.gather(
                *self.all_tracked_tasks.values(), return_exceptions=True
            )
        self.all_tracked_tasks.clear()
        if self.http_pool:
            await self.http_pool.aclose()
        if self.redis_pool:
            await self.redis_pool.disconnect()
```

4. Behavioral Governance: Operating Modes, Strategies & Scope Classifier

```

from pydantic import BaseModel, Field, DirectoryPath, model_validator
from enum import Enum
from pathlib import Path
import os

class AgentExecutionMode(str, Enum):
    PLAN = "PLAN"
    AGENT = "AGENT"

class CodeGenerationScope(str, Enum):
    INLINE_SNIPPET = "INLINE_SNIPPET"
    PATCH_PROPOSAL = "PATCH_PROPOSAL"
    FILE_MUTATION = "FILE_MUTATION"
    MULTI_FILE_CHANGESET = "MULTI_FILE_CHANGESET"

class ExecutionStrategy(str, Enum):
    DIRECT = "DIRECT"
    PLAN_AND_EXECUTE = "PLAN_AND_EXECUTE"
    REACT = "REACT"
    REFLECTION = "REFLECTION"

class RigorLevel(str, Enum):
    LOW = "LOW"
    MEDIUM = "MEDIUM"
    HIGH = "HIGH"

class ModelTier(str, Enum):
    HAIKU = "HAIKU"
    PRO = "PRO"

class ToolOperationKind(str, Enum):
    READ = "READ"
    WRITE = "WRITE"
    EXTERNAL_MUTATION = "EXTERNAL_MUTATION"

class OptimusToolErrorCode(str, Enum):
    VALIDATION_ERROR = "ERR_TOOL_VALIDATION"
    RESOURCE_ERROR = "ERR_TOOL_RESOURCE"
    PROVIDER_ERROR = "ERR_TOOL_PROVIDER"
    TIMEOUT_ERROR = "ERR_TOOL_TIMEOUT"

```

```

class OptimusToolError(Exception):
    def __init__(self, code: OptimusToolErrorCode, message: str,
                 context: dict = None):
        super().__init__(message)
        self.code = code
        self.message = message
        self.context = context or {}

class AssumptionItem(BaseModel):
    claim: str
    basis: str
    confidence: str = "medium"
    requires_verification: bool = True

class AssumptionLedger(BaseModel):
    assumptions: list[AssumptionItem] = Field(default_factory=list)

```

```

class ToolResponse(BaseModel):
    is_success: bool
    payload: dict = Field(default_factory=dict)
    error_code: Optional[OptimusToolErrorCode] = None
    error_msg: Optional[str] = None

    def to_jsonrpc_error(self) -> dict:
        code_mapping = {
            OptimusToolErrorCode.VALIDATION_ERROR: -32002,

```

```

    OptimusToolErrorCode.TIMEOUT_ERROR: -32005,
}
rpc_code = code_mapping.get(self.error_code, -32603)
return {
    "code": rpc_code,
    "message": f"Tool Execution Failure: {self.error_msg}",
    "data": {
        "tool_error_code": (
            self.error_code.value if self.error_code else None
        ),
        "context": self.payload,
    },
}

```

```

def classify_generation_scope(
    line_count: int, file_count: int, creates_or_deletes: bool,
    touches_shared: bool, directory_root_count: int,
) -> CodeGenerationScope:
    if (
        touches_shared
        or directory_root_count > 1
        or (creates_or_deletes and file_count > 1)
        or file_count > 3
    ):
        return CodeGenerationScope.MULTI_FILE_CHANGESET
    if (creates_or_deletes and file_count == 1) or (
        file_count == 1 and line_count > 100
    ):
        return CodeGenerationScope.FILE_MUTATION
    if line_count > 15:
        return CodeGenerationScope.PATCH_PROPOSAL
    return CodeGenerationScope.INLINE_SNIPPET

```

```

def select_strategy(
    task_risk: str, ambiguity: str, mutation_scope: str,
    validation_failed: bool, touches_shared_module: bool,
) -> Tuple[ExecutionStrategy, dict]:
    budget = {
        "max_tool_calls": 5,
        "max_reflection_passes": 0,
        "max_context_tokens": 100000,
        "preferred_model_tier": ModelTier.HAIKU.value,
    }
    if validation_failed or touches_shared_module:
        budget.update({
            "max_tool_calls": 15,
            "max_reflection_passes": 3,
            "max_context_tokens": 250000,
            "preferred_model_tier": ModelTier.PRO.value,
        })
    return ExecutionStrategy.REFLECTION, budget
    if task_risk == "HIGH" or mutation_scope == "MULTI_FILE_CHANGESET":
        budget.update({
            "max_tool_calls": 10,
            "max_reflection_passes": 1,
            "max_context_tokens": 150000,
            "preferred_model_tier": ModelTier.PRO.value,
        })
    return ExecutionStrategy.PLAN_AND_EXECUTE, budget
    if ambiguity == "HIGH":
        budget.update({
            "max_tool_calls": 12,
            "max_context_tokens": 150000,
            "preferred_model_tier": ModelTier.PRO.value,
        })
    return ExecutionStrategy.REACT, budget
    return ExecutionStrategy.DIRECT, budget

```

```

def assert_mutation_allowed(
    mode: AgentExecutionMode, op_kind: ToolOperationKind, operation: str,
):
    if mode == AgentExecutionMode.PLAN and op_kind in {

```

```

        raise OptimusToolError(
            OptimusToolErrorCode.VALIDATION_ERROR,
            f"Access security violation error. ",
            f"Mutation blocked in PLAN mode: {operation}",
        )

class MutationGuard:
    def __init__(self, mode: AgentExecutionMode):
        self.mode = mode

    def enforce_gate(self, op_kind: ToolOperationKind, tool_name: str):
        assert_mutation_allowed(self.mode, op_kind, tool_name)

```

```

class ASTOptimizationInput(BaseModel):
    project_path: DirectoryPath = Field(
        ..., description="Absolute path to project root."
    )
    target_file: str = Field(
        ..., description="Relative file path string."
    )
    adl_rules: list[str] = Field(default_factory=list)
    execution_mode: AgentExecutionMode = Field(
        default=AgentExecutionMode.PLAN
    )
    generation_scope: CodeGenerationScope = Field(
        default=CodeGenerationScope.INLINE_SNIPPET
    )
    rigor_budget: RigorLevel = Field(default=RigorLevel.LOW)
    assumption_ledger: AssumptionLedger = Field(
        default_factory=AssumptionLedger
    )

    @model_validator(mode="after")
    def validate_paths_and_safeguards(self) -> "ASTOptimizationInput":
        if Path(self.target_file).is_absolute():
            raise ValueError(
                "Target file must be a relative path configuration."
            )
        try:
            resolved_base = Path(
                os.path.abspath(self.project_path)
            ).resolve(strict=True)
            candidate_path = Path(os.path.abspath(
                os.path.join(str(resolved_base), self.target_file)
            ))
        except (ValueError, FileNotFoundError, RuntimeError):
            raise ValueError(
                "Malformed or invalid project root directory specification."
            )
        try:
            if candidate_path.exists() or candidate_path.is_symlink():
                resolved_target = candidate_path.resolve(strict=False)
                if os.path.commonpath(
                    [str(resolved_base), str(resolved_target)]
                ) != str(resolved_base):
                    raise ValueError(
                        "Breakout attempt detected via target file path alignment."
                    )
            else:
                parent_path = candidate_path.parent
                resolved_parent = parent_path.resolve(strict=True)
                if os.path.commonpath(
                    [str(resolved_base), str(resolved_parent)]
                ) != str(resolved_base):
                    raise ValueError(
                        "Symlink ancestry breakout attempt detected via target parent folder."
                    )
        except ValueError as ve:
            raise ValueError(
                f"Cross-drive partition boundary constraint or layout failure: {str(ve)}"
            )
        except (FileNotFoundError, RuntimeError):
            raise ValueError(
                "Missing or invalid ancestral directory infrastructure framework structure mapping."
            )
        return self

```

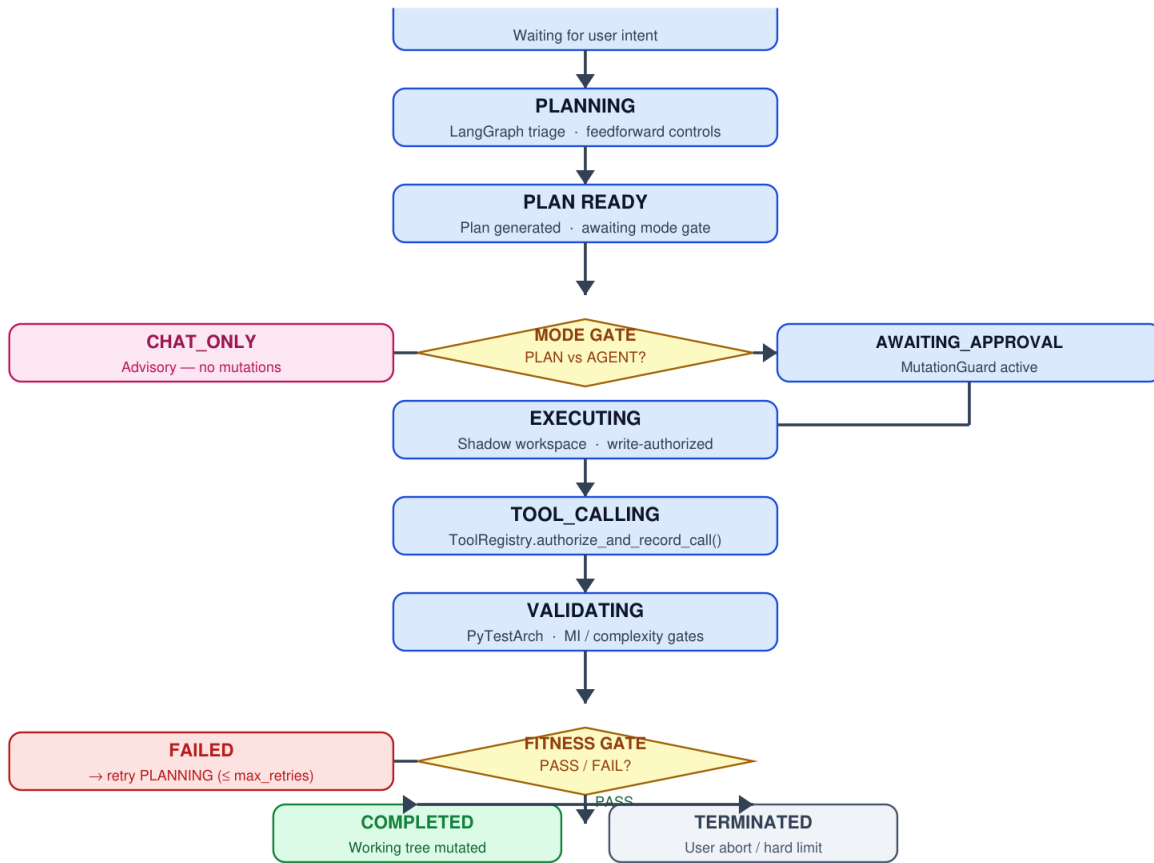
4A. Agent State Model and Execution Pathways

This section defines the complete agent lifecycle — from initial idle state through planning, execution, tool calling, and termination — and specifies which transitions are valid, which are rejected deterministically, and what permissions each mode grants.

Lifecycle State Machine

The agent operates as a deterministic state machine. Every state transition is validated at the control-plane boundary; invalid transitions are rejected with a structured -32002 error before any side-effectful work begins.

Section 4A · Agent Lifecycle State Machine



Deterministic state machine — every transition validated at the mutation boundary (MutationGuard).

Valid and Invalid Transition Table

Every state pair is enumerated below. The harness enforces these deterministically — no model instruction can override a rejected transition.

From	To	Permitted	Condition / Rejection Reason
Idle	Planning	Yes	Always — triggered by any user request.
Planning	PlanReady	Yes	Requirements clarified; mode selected.
PlanReady	ChatOnly	Yes	mode = Plan/Chat; no mutation allowed.
PlanReady	Executing (direct)	No	Rejected. Must pass through AwaitingApproval.
PlanReady	AwaitingApproval	Yes	mode = Agent; awaits explicit user approval.
AwaitingApproval	Executing	Yes	Approval granted within timeout window.
AwaitingApproval	ChatOnly	Yes	Approval denied or timeout; falls back advisory.
Executing	ToolCalling	Yes	Tool invocation authorised by ToolInvocationPolicy.
Executing	ToolCalling (mutation)	Cond.	Mutation tools blocked if mode != Agent.
ToolCalling	Validating	Yes	Tool response received; fitness gates run.
Validating	Executing	Yes	All gates passed.
Validating	Failed	Yes	Any gate fails; retry counter incremented.
Failed	Planning	Yes	Retry budget not exhausted (max 3 retries).
Failed	Terminated	Yes	Budget exhausted; user escalation emitted.
Executing	Completed	Yes	All planned work done; no open gates.
Any	Executing (bypass)	No	Rejected -32002. No path bypasses AwaitingApproval.

Mode-Specific Permission Matrix

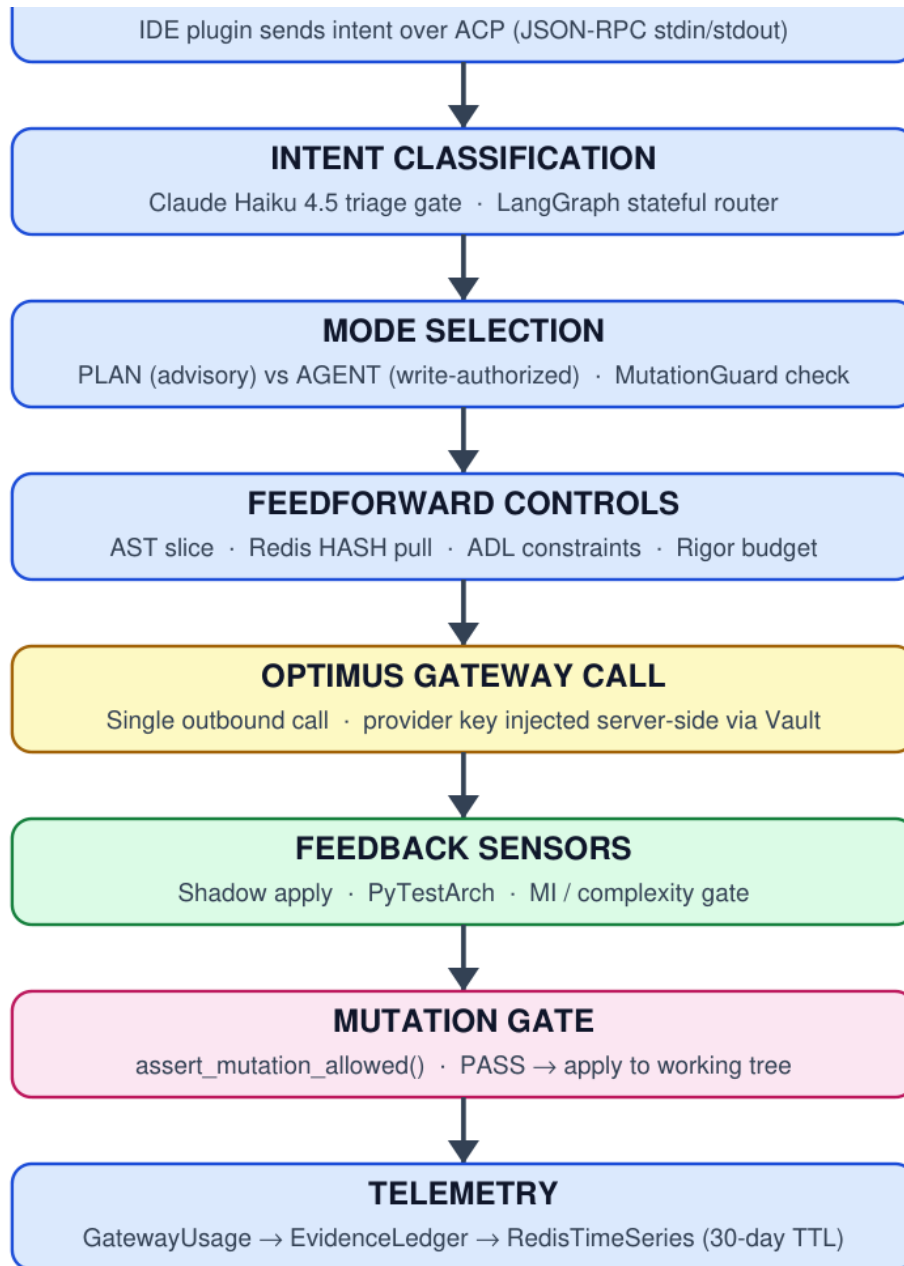
Permissions are enforced at the harness layer, not by model instruction. Any attempt to invoke a mutation tool outside Agent mode is rejected before the tool call is dispatched.

Capability	Plan/Chat	Agent	Enforcement
Read repo / inspect files	Yes	Yes	Always allowed.
Run read-only analysis tools	Yes	Yes	Always allowed.
Web search / extract (policy-gated)	Yes	Yes	ToolInvocationPolicy must approve.
Produce diffs / snippets as text	Yes	Yes	Returned as text payload, not applied.
Shadow-apply patch (read-only test)	No	Yes	assert_mutation_allowed() gate.
Mutate working tree (write files)	No	Yes	assert_mutation_allowed() + MutationGuard.
Run shell / build / test	No	Yes	Agent mode + composite fitness gate.
Append cost / audit telemetry	Yes	Yes	Always-on; append-only.
Modify telemetry / ledger entries	No	No	Immutable after write.

Execution Pathway

The execution pathway shows how a user request flows through intent classification, mode selection, mutation gating, and telemetry — with the Optimus Gateway as the single external call point.

Section 4A · Execution Pathway



End-to-end execution pathway. Providers (GLM / Haiku / Sonnet / Opus) are selected by the Gateway via model alias.

Execution Strategy Selection

The agent selects one of four execution strategies based on task ambiguity, mutation risk, and prior validation outcomes. Strategy selection is harness-controlled, not model-controlled.

Strategy	Trigger	Behaviour	Cost Profile
Direct Execution	LOW rigor, no mutations	Single model call; apply result; no reflection.	<=5 tool calls, 0 reflections, Haiku.
Plan-and-Execute	MEDIUM rigor, mutation required	Structured plan; execute step-by-step; validate each step.	<=10 tool calls, 1 reflection, Pro.
ReAct	HIGH rigor, ambiguous/multi-file	Interleave reasoning and tool calls; update plan per observation.	<=15 tool calls, up to 3 reflections, Pro.
Reflection	Exception: gate failure or shared-core	Triggered by failed fitness gate or MULTI_FILE_CHANGESSET. Not a default loop.	Bounded to 3 passes max.

Tool Invocation Decision Rules

Tool invocation is always harness-controlled. The model may express intent to call a tool; the ToolInvocationPolicy decides whether it is permitted. Rules are evaluated in priority order; first match wins.

```
# ToolInvocationPolicy evaluation order (first match wins)

if mode == PLAN_CHAT and tool.is_mutation:
    REJECT # -32002: mutation tools forbidden in Plan/Chat mode

if tool.class == WEB_SEARCH:
    if not any([user_requested_external_fact, task_mentions_unknown_api,
                task_touches_security_cves]):
        REJECT # local evidence first; no casual web calls
    require reason_code in VALID_REASON_CODES
    ALLOW with EvidenceRequest(query=verbatim, reason=reason_code)

if tool.class == WEB_EXTRACT:
    require url in prior_search_result_set # provenance check
    ALLOW with EvidenceExtractRequest(url=url)

if tool.class == MUTATION and not approval_granted:
    REJECT # -32002: no mutation before AwaitingApproval clears

DEFAULT: ALLOW # local read/inspect/analysis tools
```

Failure and Retry Lifecycle

When a fitness gate fails, the agent records the failure signal, increments the retry counter, and re-enters Planning with failure context injected into the next prompt. After three consecutive failures, execution terminates and the user is escalated with a structured failure report.

```
class RetryPolicy:
    max_retries: int = 3
    backoff_base_ms: int = 500
    retry_on: tuple = (TransientGatewayError, ProviderRateLimitError)
    abort_on: tuple = (PermanentGatewayError, PolicyViolationError,
                      BudgetExhaustedError)

    def on_gate_failure(self, failure: FitnessGateFailure) -> RetryAction:
        self.failure_log.append(failure)
        if self.retry_count >= self.max_retries:
            return RetryAction.ESCALATE_TO_USER
        if failure.is_permanent:
            return RetryAction.ABORT_WITH_REPORT
        self.retry_count += 1
        return RetryAction.REPLAN_WITH_FAILURE_CONTEXT
```

5. ADL Parser Continuity & Architectural Evolution

```
from abc import ABC, abstractmethod

class ADLParserStrategy(ABC):
    @abstractmethod
    def parse_and_validate(
        self, rules: list, patch_diff: str
    ) -> Tuple[bool, list]:
        pass

class PrototypeRegexADLParser(ADLParserStrategy):
    def __init__(self):
        # Aligned with single escapes inside a raw string
        # configuration to match input streams correctly.
        self.dep_pattern = re.compile(
            r"ASSERT\(((?P<la>\w+)\s)+HAS\s+NO\s+DEPENDENCY"
            r"\s+ON\s+(?P<lb>\w+)\s)"
        )

    def parse_and_validate(
        self, rules: list, patch_diff: str
    ) -> Tuple[bool, list]:
        violations = []
        for rule in rules:
            match = self.dep_pattern.match(rule)
            if not match:
                continue
            la, lb = match.group("la"), match.group("lb")
            lb_lower = lb.lower()
            has_import = f"import {lb_lower}" in patch_diff
            has_spring_dep = (
                f"org.springframework.{lb_lower}" in patch_diff
            )
            if has_import or has_spring_dep:
                violations.append({
                    "rule": rule,
                    "metric": "Structural Pollution",
                    "value": f"Violation: {la}->{lb}",
                })
        return len(violations) == 0, violations
```

6. Resilient Provider Calling Layer & Tenacity Rules

All model completions route through the Optimus Gateway using the single Optimus credential. The local agent never calls an upstream LLM provider directly and never holds a provider API key. The gateway resolves the model alias, selects the upstream provider, injects its own Vault-held key, and returns a normalised response.

The gateway exposes two completion endpoints with distinct wire shapes. Use `/v1/responses` for the OpenAI Responses API shape, where the request body uses "input" as the top-level content field. Use `/v1/chat/completions` for the Chat Completions shape, where the request body uses a "messages" array. The local agent must not mix these: sending a "messages" array to `/v1/responses`, or an "input" string to `/v1/chat/completions`, will be rejected by the gateway schema validator. The example below uses `/v1/responses` with the Responses API shape.

```
from tenacity import (
    retry, stop_after_attempt, wait_random_exponential,
    retry_if_exception,
)

def is_retryable_provider_fault(exc: Exception) -> bool:
    is_timeout = isinstance(exc, asyncio.TimeoutError)
    is_tool_fault = isinstance(exc, OptimusToolError) and exc.code in {
        OptimusToolErrorCode.PROVIDER_ERROR,
        OptimusToolErrorCode.TIMEOUT_ERROR,
    }
    return is_timeout or is_tool_fault

def log_retry_telemetry(state(retry_state):
    if retry_state and isinstance(retry_state.args[0], dict):
        telemetry = retry_state.args[0].setdefault("telemetry", {})
        telemetry["retry_count"] = retry_state.attempt_number
```

```
@retry(
    reraise=True,
    stop=stop_after_attempt(4),
    wait=wait_random_exponential(min=1, max=12),
    retry=retry_if_exception(is_retryable_provider_fault),
    after=log_retry_telemetry_state,
)

async def call_frontier_model_provider_resiliently(
    state: dict,
    shared_http_client: httpx.AsyncClient,
    gateway_settings: OptimusGatewaySettings,
) -> dict:
    """Routes through the Optimus Gateway (POST /v1/responses).
    No provider URL or provider API key is present here.
    The gateway selects the upstream model, injects its Vault
    credentials, and returns a normalised completion response."""
    telemetry = state.setdefault("telemetry", {"retry_count": 0})
    if telemetry.get("retry_count", 0) < 2:
        raise OptimusToolError(
            OptimusToolErrorCode.PROVIDER_ERROR,
            "Rate limit triggered (HTTP 429)",
        )
    # POST to the gateway Responses endpoint.
    # The OpenAI Responses API (as opposed to Chat Completions) uses
    # "input" as the top-level request field, not "messages". The gateway
    # exposes this shape at /v1/responses. If you need Chat Completions
    # wire format, use /v1/chat/completions with "messages" instead.
    gateway_url = f"{gateway_settings.gateway_url}/v1/responses"
    payload = {
        "model": state.get("model_alias", "optimus-default"),
        "input": state.get("optimized_payload", ""), # Responses API field
    }
    try:
        response = await shared_http_client.post(
            gateway_url,
            json=payload,
```

```
if response.status_code != 200:
    raise OptimusToolError(
        OptimusToolErrorCode.PROVIDER_ERROR,
        f"Gateway returned non-200: {response.status_code}",
    )
body = response.json()
return {
    "status": "SUCCESS",
    "generated_patch": body.get("patch", ""),
    "gateway_request_id": body.get("gateway_request_id", ""),
    "provider": body.get("provider", ""),
    "cache_hit": body.get("cache_hit", False),
}
except httpx.HTTPError as h_err:
    raise OptimusToolError(
        OptimusToolErrorCode.PROVIDER_ERROR,
        f"Gateway communication failure: {str(h_err)}",
    )
```

7. Composite Fitness Engine Scaffolding

```

class FitnessGateInterface(ABC):
    @abstractmethod
    def evaluate_gate(self, state: dict) -> Tuple[bool, dict]:
        pass

class DependencyRuleGate(FitnessGateInterface):
    def __init__(self, parser: ADLParserStrategy):
        self.parser = parser

    def evaluate_gate(self, state: dict) -> Tuple[bool, dict]:
        passed, violations = self.parser.parse_and_validate(
            state.get("adl_rules", []), state.get("generated_patch", "")
        )
        return passed, {"violations": violations}

class PlaceholderMetricGate(FitnessGateInterface):
    def __init__(self, max_complexity: int = 15):
        self.max_complexity = max_complexity

    def evaluate_gate(self, state: dict) -> Tuple[bool, dict]:
        return True, {
            "metrics": {
                "cyclomatic_complexity": 4,
                "maintainability_index": 82,
            }
        }

class PlaceholderTestArchGate(FitnessGateInterface):
    def evaluate_gate(self, state: dict) -> Tuple[bool, dict]:
        return True, {
            "testarch_status": "COMPLIANT",
            "graph_cyclical": False,
        }

class CompositeFitnessEngine:
    def __init__(self, gates: list[FitnessGateInterface]):
        self.gates = gates

    def execute_harness_check(self, state: dict) -> dict:
        combined_telemetry = {}
        for gate in self.gates:
            passed, telemetry_delta = gate.evaluate_gate(state)
            combined_telemetry.update(telemetry_delta)
            if not passed:
                return {
                    "fitness_verdict": "FAIL",
                    "telemetry": combined_telemetry,
                }
        return {
            "fitness_verdict": "PASS",
            "telemetry": combined_telemetry,
        }

```

8. Patch Workspace Lifecycle Boundaries

Applying patches exposes systems to directory traversal attacks. Optimus normalises boundaries using strict commonpath comparisons and strictly isolates validation steps entirely inside a localised shadow workspace environment.

- **Shadow Workspace Environment:** Patches are written exclusively to a workspace-local shadow directory located at `.optimus/shadow_env/`. The patch diff is programmatically parsed, applied, and validated against the duplicate code tree via python file mutation loops.
- **Operating Mode Enforcement Contract:** If `execution_mode` equals `AgentExecutionMode.PLAN`, tool operations matching `ToolOperationKind.WRITE` or `ToolOperationKind.EXTERNAL_MUTATION` are completely bypassed and blocked from modifying local structures. Read-only inspection tools remain fully authorised.
- **Atomic Apply to Working Tree & Surfacing Rejections:** Modifications are applied to the working tree only after a clean PASS verdict, guaranteeing transaction atomicity and immediate rollback safety on any system fault. If a gate trips a FAIL signature, the patch is rejected, the transaction aborts, and the duplicate shadow environment is cleanly purged.

9. Tool Registry, Invocation Policy & Evidence Ledger

Tool use is policy-driven first, model-requested second: the model may request evidence, but the harness alone decides whether a tool is permitted, necessary, and cost-justified for the active execution mode and rigor tier. This section defines the typed tool registry, the deterministic invocation policy matrix, and the Evidence Ledger schema that records every external lookup alongside the Assumption Ledger.

A. Tool Class Enum & Registry Contract

```
class ToolClass(str, Enum):
    LOCAL_REPO_DISCOVERY = "LOCAL_REPO_DISCOVERY"
    VALIDATION_GATE = "VALIDATION_GATE"
    PATCH_WORKSPACE = "PATCH_WORKSPACE"
    COST_TELEMETRY = "COST_TELEMETRY"
    WEB_EVIDENCE = "WEB_EVIDENCE"
    PACKAGE_AND_ADVISORY_METADATA = "PACKAGE_AND_ADVISORY_METADATA"
    EXTERNAL_EXECUTION = "EXTERNAL_EXECUTION"

class EvidenceReasonCode(str, Enum):
    API_DOCS_OUTDATED = "API_DOCS_OUTDATED"
    CURRENT_FACT = "CURRENT_FACT"
    SECURITY_ADVISORY = "SECURITY_ADVISORY"
    USER_REQUESTED = "USER_REQUESTED"
    PACKAGE_VERSION = "PACKAGE_VERSION"

class SearchDepth(str, Enum):
    """Internal rigor presets. NOT assumed to equal the gateway's wire
    values 1:1 -- see TAVILY_SEARCH_DEPTH_MAP in section C, which is
    the only place an internal value is translated into the string
    actually sent to the provider via the gateway."""
    BASIC = "basic"
    FAST = "fast"
    ADVANCED = "advanced"

class ToolRegistryEntry(BaseModel):
    tool_name: str
    tool_class: ToolClass
    allowed_modes: list[AgentExecutionMode]
    requires_policy_trigger: bool = False
    is_mutating: bool = False
    max_calls_per_run: Optional[int] = None

class ToolRegistry(BaseModel):
    entries: dict[str, ToolRegistryEntry] = Field(default_factory=dict)
    # In-process call counters, keyed by f"{run_id}:{tool_name}". Single
    # process / single-instance only -- see the Redis-backed variant
    # below for concurrent or multi-process execution.
    call_counts: dict[str, int] = Field(default_factory=dict)

    def check_static_policy(
        self, entry: Optional[ToolRegistryEntry],
        mode: AgentExecutionMode, policy_triggered: bool,
    ) -> bool:
        if entry is None:
            return False
        if mode not in entry.allowed_modes:
            return False
        if entry.requires_policy_trigger and not policy_triggered:
            return False
        return True

    def is_allowed(
        self, tool_name: str, mode: AgentExecutionMode,
        policy_triggered: bool, run_id: str,
    ) -> bool:
        """READ-ONLY inspection (e.g. UI previews, dry-run explainers).
        Does not increment the call counter. Never use this to gate an
        actual tool invocation -- use authorize and record call() below,
```

```

    entry = self.entries.get(tool_name)
    if not self.check_static_policy(entry, mode, policy_triggered):
        return False
    if entry.max_calls_per_run is not None:
        counter_key = f"{run_id}:{tool_name}"
        calls_so_far = self.call_counts.get(counter_key, 0)
        if calls_so_far >= entry.max_calls_per_run:
            return False
        return True

def authorize_and_record_call(
    self, tool_name: str, mode: AgentExecutionMode,
    policy_triggered: bool, run_id: str,
) -> bool:
    """The only method that should gate an actual tool call. Checks
    the static policy and the call-count ceiling, then increments
    the counter, as a single atomic step -- so authorization and
    bookkeeping can never drift apart, and a caller cannot forget
    to record a call it was just allowed to make."""
    entry = self.entries.get(tool_name)
    if not self.check_static_policy(entry, mode, policy_triggered):
        return False
    counter_key = f"{run_id}:{tool_name}"
    if entry.max_calls_per_run is not None:
        calls_so_far = self.call_counts.get(counter_key, 0)
        if calls_so_far >= entry.max_calls_per_run:
            return False
    self.call_counts[counter_key] = (
        self.call_counts.get(counter_key, 0) + 1
    )
    return True

```

The in-process dict above is sufficient for Phase 1's single-instance daemon, but is not safe across multiple worker processes and does not survive a restart mid-run. The production-grade path swaps the in-memory counter for a Redis-backed atomic increment, keyed identically and protected by a TTL matching the run's expected lifetime:

```

async def authorize_and_record_call_redis(
    registry: ToolRegistry, redis_client: aioredis.Redis,
    tool_name: str, mode: AgentExecutionMode,
    policy_triggered: bool, run_id: str, run_ttl_secs: int = 3600,
) -> bool:
    entry = registry.entries.get(tool_name)
    if not registry.check_static_policy(entry, mode, policy_triggered):
        return False
    if entry.max_calls_per_run is None:
        return True
    counter_key = f"toolcalls:{run_id}:{tool_name}"
    # INCR is atomic at the Redis level, so concurrent callers cannot
    # race past the ceiling the way two in-process checks could.
    new_count = await redis_client.incr(counter_key)
    if new_count == 1:
        await redis_client.expire(counter_key, run_ttl_secs)
    if new_count > entry.max_calls_per_run:
        await redis_client.decr(counter_key)
        return False
    return True

```

B. Deterministic Invocation Policy Matrix

The policy matrix maps task signals to permitted tool classes. The model never selects a tool class directly; it raises a typed signal and the harness resolves the matching policy row.

PACKAGE_AND_ADVISORY_METADATA deliberately covers both package-registry lookups (PyPI, npm, Maven Central) and security-advisory lookups (OSV.dev, GitHub advisories): both are read-only, policy-triggered, and keyed by a package or CVE identifier, so a single tool class keeps the registry simple while the signal name (TASK_TOUCHES_DEPENDENCIES vs. TASK_TOUCHES_SECURITY_CVE) still makes the routing intent explicit.

```

class ToolInvocationPolicy(BaseModel):
    signal: str
    permitted_class: ToolClass
    requires_reason_code: bool = False

DEFAULT_POLICY_MATRIX: list[ToolInvocationPolicy] = [
    ToolInvocationPolicy(
        signal="USER REQUESTS CURRENT FACT",
        permitted_class=ToolClass.WEB_EVIDENCE,
        requires_reason_code=True,
    ),
    ToolInvocationPolicy(
        signal="LIBRARY BEHAVIOR NOT IN REPO",
        permitted_class=ToolClass.WEB_EVIDENCE,
        requires_reason_code=True,
    ),
    ToolInvocationPolicy(
        signal="TASK TOUCHES DEPENDENCIES",
        permitted_class=ToolClass.PACKAGE_AND_ADVISORY_METADATA,
        requires_reason_code=True,
    ),
    ToolInvocationPolicy(
        signal="TASK TOUCHES SECURITY CVE",
        permitted_class=ToolClass.PACKAGE_AND_ADVISORY_METADATA,
        requires_reason_code=True,
    ),
    ToolInvocationPolicy(
        signal="LOCAL CODE CHANGE REQUESTED",
        permitted_class=ToolClass.LOCAL_REPO_DISCOVERY,
        requires_reason_code=False,
    ),
    ToolInvocationPolicy(
        signal="UNVERIFIED ARCHITECTURAL CLAIM",
        permitted_class=ToolClass.LOCAL_REPO_DISCOVERY,
        requires_reason_code=False,
    ),
    ToolInvocationPolicy(
        signal="PATCH GENERATED",
        permitted_class=ToolClass.PATCH_WORKSPACE,
        requires_reason_code=False,
    ),
]

class EvidenceRequest(BaseModel):
    query: str
    need_external_evidence: bool = True
    reason: EvidenceReasonCode
    allowed_domains: list[str] = Field(default_factory=list)
    max_results: int = Field(default=5, le=10)
    search_depth: SearchDepth = SearchDepth.BASIC

    @model_validator(mode="after")
    def validate_rigor_alignment(self) -> "EvidenceRequest":
        if not self.query.strip():
            raise ValueError(
                "query must be a non-empty search string; reason is "
                "metadata only and must never be sent as the query."
            )
        if self.search_depth == SearchDepth.ADVANCED and self.max_results > 5:
            raise ValueError(
                "Advanced search depth is capped at 5 results to bound "
                "per-call credit cost."
            )
        return self

```

C. Typed Evidence Acquisition Wrappers

In the current Gateway-centric architecture, the local agent no longer holds any provider API key. Authentication to all downstream services (Tavily, OSV, package registries) is handled by the Optimus AI Gateway. The local agent supplies only its Optimus credential; the gateway resolves the provider route and injects its own Vault-held keys server-side. This eliminates credential sprawl on developer machines and centralises cost attribution.

The developer-facing configuration model is now OptimusGatewaySettings. The legacy TavilySettings model (with api_key: SecretStr) is promoted to a server-side-only concern, documented as GatewayProviderSecrets; it belongs in gateway / admin / internal design, not in the local agent configuration path.

```
from pydantic import SecretStr, model_validator as mv

# — Developer-facing config (local agent LLD) —————
# The ONLY credentials a developer configures. Provider keys (Tavily,
# OpenAI, etc.) are held server-side in the gateway Vault and are never
# seen by the local agent.

# Built-in trusted origins. These cover Optimus-hosted regions and are
# the only origins valid in production without a tenant profile.
OPTIMUS_BUILTIN_TRUSTED_ORIGINS: frozenset[str] = frozenset({
    "https://gateway.optimus.ai", # Optimus-hosted production
    "https://gateway.optimus-eu.ai", # Optimus-hosted EU region
})

# — Origin trust model —————
# Production mode: extra origins come ONLY from a signed tenant profile
# (installer bootstrap or admin-controlled config). The Optimus credential
# must never be sent to an origin that the local process can choose freely.
#
# Dev/test mode: OPTIMUS_EXTRA_GATEWAY_ORIGINS env var is accepted to
# allow local gateway stubs. This env var MUST be blocked at production
# build time (e.g., stripped by the installer or rejected when
# OPTIMUS_PRODUCTION_MODE=true).

def load_trusted_origins(production_mode: bool = False) -> frozenset[str]:
    """Return the set of origins the local agent may send credentials to.
    In production mode, only built-in origins + signed tenant profile.
    In dev/test mode, env var extras are also accepted."""
    if production_mode:
        # Tenant profile is injected by the installer/admin tool at
        # bootstrap time and is not modifiable by the local process.
        tenant_extras = read_signed_tenant_profile_origins()
        return OPTIMUS_BUILTIN_TRUSTED_ORIGINS | tenant_extras
    extras = os.getenv("OPTIMUS_EXTRA_GATEWAY_ORIGINS", "")
    extra_set = frozenset(o.strip() for o in extras.split(",") if o.strip())
    return OPTIMUS_BUILTIN_TRUSTED_ORIGINS | extra_set

def read_signed_tenant_profile_origins() -> frozenset[str]:
    """Placeholder: read and verify the cryptographically-signed tenant
    profile written by the Optimus installer or enterprise admin tool.
    Returns additional trusted gateway origins for this org deployment.
    Implementation lives in the admin/installer layer, not in LLD scope."""
    return frozenset()

class OptimusGatewaySettings(BaseModel):
    gateway_url: str = "https://gateway.optimus.ai"
    optimus_api_key: SecretStr
    production_mode: bool = False
    # extra_trusted_origins for dev/test only; ignored when
    # production_mode=True (tenant profile is the authoritative source).
    extra_trusted_origins: frozenset[str] = Field(default_factory=frozenset)

    @model_validator(mode="after")
    def validate_trusted_gateway(self) -> "OptimusGatewaySettings":
        if self.production_mode and self.extra_trusted_origins:
            raise ValueError(
                "extra_trusted_origins must not be set in production_mode. "
                "Enterprise gateways must be registered via the signed tenant "
                "profile written by the Optimus installer or admin tool.",
            )
        trusted = (load_trusted_origins(self.production_mode)
            | (frozenset() if self.production_mode
            else self.extra_trusted_origins))
        if self.gateway_url not in trusted:
            raise ValueError(
                f"gateway_url '{self.gateway_url}' is not in the trusted origin",
                "set. In production, register it via the signed tenant profile.",
                "In dev/test, add it to OPTIMUS_EXTRA_GATEWAY_ORIGINS or",
                "extra_trusted_origins.",
            )
        return self
```

```

return {
    "Authorization": f"Bearer {self.optimus_api_key.get_secret_value()}"
}

```

```

# — Server-side only: gateway / admin / internal design —
# GatewayProviderSecrets NEVER appears in the local agent configuration.
# It is documented here for completeness; the gateway Vault owns these
# values and maps them from the authenticated Optimus credential.

```

```

class GatewayProviderSecrets(BaseModel):
    openai_api_key: SecretStr | None = None
    openrouter_api_key: SecretStr | None = None
    tavily_api_key: SecretStr | None = None

```

SearchDepth is an internal rigor preset, not a gateway wire value. It is translated explicitly before the request leaves the process so that a future change to the gateway's accepted values only requires updating one map, not every call site:

```

# BASIC and FAST both currently map to the gateway's "basic" depth.
# FAST is a distinct internal preset reserved for future rigor-tier
# tuning, without assuming its string form is safe to send as-is.
TAVILY_SEARCH_DEPTH_MAP: dict[SearchDepth, str] = {
    SearchDepth.BASIC: "basic",
    SearchDepth.FAST: "basic",
    SearchDepth.ADVANCED: "advanced",
}

```

```

# — Gateway usage accounting —
# Every gateway response carries a structured usage envelope.
# All fields are recorded on the Evidence Ledger for cost attribution.

```

```

class GatewayUsage(BaseModel):
    gateway_request_id: str
    provider: str # e.g. "tavily", "openai", "openrouter"
    provider_request_id: str | None = None
    cache_hit: bool = False
    billing_units: int = 0 # provider-native unit (credits/tokens)
    cost_usd: float = 0.0 # gateway-computed USD cost

```

```

class WebEvidenceResult(BaseModel):
    url: str
    title: str
    snippet: str
    score: float

```

```

class WebEvidenceResponse(BaseModel):
    results: list[WebEvidenceResult]
    credits_used: int = 0 # kept for backward compat
    gateway_usage: GatewayUsage | None = None

```

```

async def search_web_evidence(
    request: EvidenceRequest, settings: OptimusGatewaySettings,
    shared_http_client: httpx.AsyncClient,
) -> WebEvidenceResponse:
    """The local agent calls the Optimus Gateway, not Tavily directly.
    The gateway proxies to Tavily using its own Vault-held key and
    returns a normalised response, surfacing credits_used for the
    Evidence Ledger."""
    if not request.need_external_evidence:
        raise OptimusToolError(
            OptimusToolErrorCode.VALIDATION_ERROR,
            "Evidence request declared need_external_evidence=False.",
        )
    payload = {
        "query": request.query,
        "search_depth": TAVILY_SEARCH_DEPTH_MAP[request.search_depth],
        "max_results": request.max_results,
        "include_domains": request.allowed_domains,
    }

```

```

        f"{settings.gateway_url}/v1/tools/web/search",
        json=payload,
        headers=settings.auth_headers(),
    )
    response.raise_for_status()
except httpx.HTTPError as h_err:
    raise OptimusToolError(
        OptimusToolErrorCode.PROVIDER_ERROR,
        f"Gateway web search failure: {str(h_err)}",
    )
body = response.json()
results = [
    WebEvidenceResult(
        url=r.get("url", ""), title=r.get("title", ""),
        snippet=r.get("content", ""), score=r.get("score", 0.0),
    )
    for r in body.get("results", [])
]
usage_env = body.get("usage", {})
gw_usage = GatewayUsage(
    gateway_request_id = body.get("gateway request id", ""),
    provider = usage_env.get("provider", "tavily"),
    provider_request_id = usage_env.get("provider request id"),
    cache_hit = usage_env.get("cache hit", False),
    billing_units = usage_env.get("credits", 0),
    cost_usd = usage_env.get("cost usd", 0.0),
)
if body.get("gateway_request_id") else None
return WebEvidenceResponse(
    results=results,
    credits_used=usage_env.get("credits", 0),
    gateway_usage=gw_usage,
)

```

Extract previously had weaker bounds than Search: arbitrary URLs and an unbounded `max_chars_per_source` could be passed straight through. `EvidenceExtractRequest` below bounds both, and `extract_web_sources()` requires every URL to already appear in a prior approved search response — Extract is a follow-up on evidence the harness has already seen, never an independent fetch of arbitrary URLs. The call is proxied through the gateway, matching the pattern established for Search.

```

class EvidenceExtractRequest(BaseModel):
    urls: list[str] = Field(..., min_length=1, max_length=10)
    max_chars_per_source: int = Field(default=4000, gt=0, le=20000)
    rerank_query: Optional[str] = None

    @model_validator(mode="after")
    def validate_extract_bounds(self) -> "EvidenceExtractRequest":
        if len(set(self.urls)) != len(self.urls):
            raise ValueError("urls must not contain duplicates.")
        return self

class WebExtractResponse(BaseModel):
    content_by_url: dict[str, str]
    credits_used: int = 0 # kept for backward compat
    gateway_usage: GatewayUsage | None = None

async def extract_web_sources(
    request: EvidenceExtractRequest, approved_urls: set[str],
    settings: OptimusGatewaySettings,
    shared_http_client: httpx.AsyncClient,
) -> WebExtractResponse:
    # approved_urls is typically {r.url for r in a prior
    # WebEvidenceResponse.results} -- Extract may only retrieve sources
    # the harness already discovered via search_web_evidence(), never
    # an arbitrary caller-supplied URL.
    unapproved = [u for u in request.urls if u not in approved_urls]
    if unapproved:
        raise OptimusToolError(
            OptimusToolErrorCode.VALIDATION_ERROR,
            "Extract requested for URLs outside the prior approved "
            f"search results: {unapproved}",
        )
    payload = {"urls": request.urls}

```

```

# extracted chunks by relevance, reducing irrelevant content
# pulled into the prompt window and bounding token cost.
payload["query"] = request.rerank_query
try:
    response = await shared_http_client.post(
        f"{settings.gateway_url}/v1/tools/web/extract",
        json=payload,
        headers=settings.auth_headers(),
    )
    response.raise_for_status()
except httpx.HTTPError as h_err:
    raise OptimusToolError(
        OptimusToolErrorCode.PROVIDER_ERROR,
        f"Gateway web extract failure: {str(h_err)}",
    )
body = response.json()
truncated: dict[str, str] = {}
for item in body.get("results", []):
    url = item.get("url", "")
    content = item.get("raw_content", "") or ""
    truncated[url] = content[:request.max_chars_per_source]
usage_env = body.get("usage", {})
gw_usage = GatewayUsage(
    gateway_request_id = body.get("gateway_request_id", ""),
    provider = usage_env.get("provider", "tavily"),
    provider_request_id = usage_env.get("provider_request_id"),
    cache_hit = usage_env.get("cache_hit", False),
    billing_units = usage_env.get("credits", 0),
    cost_usd = usage_env.get("cost_usd", 0.0),
)
if body.get("gateway_request_id") else None
return WebExtractResponse(
    content by url=truncated,
    credits_used=usage_env.get("credits", 0),
    gateway_usage=gw_usage,
)

```

D. Gateway Server-Side Policy Revalidation

The gateway must not trust the local agent's pre-validated parameters. Defense in depth requires the gateway to independently revalidate every policy constraint, regardless of what the local agent sends. The local ToolRegistry and EvidenceRequest validators are client-side convenience gates; the gateway is the authoritative enforcement point.

- **Allowed domains:** the gateway re-applies the org/project domain whitelist against the fully-resolved request URL before dispatching to the upstream provider. A misconfigured or malicious local agent cannot bypass this.
- **Extract URL provenance:** the gateway re-checks that all extract URLs appeared in a preceding gateway-logged search response for the same run_id. The local agent's approved_urls set is advisory only.
- **Budgets:** the gateway re-enforces per-org, per-user, and per-project spend caps against its Cost Ledger before dispatching any billable request. Local budget state is informational.
- **Call caps:** the gateway independently tracks max_calls_per_run against its own Redis counter keyed by run_id + tool_name. It does not trust the local ToolRegistry counter.
- **Tool policies:** the gateway re-checks that the requested tool class and model are permitted for the authenticated org / project / execution mode. Plan-mode constraints are enforced server-side.

Any request that fails a gateway-side policy check returns a structured error with a gateway_request_id, allowing the local agent to surface the rejection through the normal ToolResponse error path. The gateway never silently drops requests.

E. Evidence Ledger Schema

Every external lookup is recorded in an Evidence Ledger entry alongside the existing Assumption Ledger, so a reviewer can trace which architectural claims were backed by which external source, under which policy signal and reason code. All gateway usage fields (gateway_request_id, provider, cache_hit, billing_units, cost_usd) are populated directly from the gateway response envelope, never estimated after the fact. EvidenceLedger exposes three reconciliation methods: total_credits() for backward compatibility with v2.26 callers, total_billing_units() for provider-native unit reconciliation, and total_cost_usd() as the primary cost reconciliation field (since v2.29).

```
class EvidenceLedgerEntry(BaseModel):
    reason: EvidenceReasonCode
    policy_signal: str
    tool_class: ToolClass
    queried_at: str
    sources: list[str] = Field(default_factory=list)
    # Legacy credit counter (kept for backward compat)
    credits_used: int = 0
    # Gateway usage envelope – populated when gateway_usage is present
    gateway_request_id: str = ""
    provider: str = ""
    provider_request_id: str | None = None
    cache_hit: bool = False
    billing_units: int = 0
    cost_usd: float = 0.0

class EvidenceLedger(BaseModel):
    entries: list[EvidenceLedgerEntry] = Field(default_factory=list)
    def record(self, entry: EvidenceLedgerEntry) -> None:
        self.entries.append(entry)
    def total_credits(self) -> int:
        """Legacy credit total – backward compat with v2.26 callers."""
        return sum(e.credits_used for e in self.entries)
    def total_billing_units(self) -> int:
        """Sum of provider-native billing units across all entries.
        Reconciles against the gateway's own billing_units accounting."""
        return sum(e.billing_units for e in self.entries)
    def total_cost_usd(self) -> float:
        """Sum of gateway-computed USD cost across all entries.
        Primary reconciliation field since v2.29; total_credits()
        is retained for backward compatibility only."""
        return sum(e.cost_usd for e in self.entries)

# Example wiring: search_web_evidence() result feeds the ledger entry
# directly. All gateway usage fields are populated from the response,
# never estimated after the fact.
response = await search_web_evidence(request, settings, shared_http_client)
gu = response.gateway_usage # GatewayUsage | None
evidence_ledger.record(EvidenceLedgerEntry(
    reason=request.reason,
    policy_signal=policy_signal,
    tool_class=ToolClass.WEB_EVIDENCE,
    queried_at=datetime.now(timezone.utc).isoformat(),
```

```

sources=[r.url for r in response.results],
credits_used=response.credits_used,
gateway_request_id = gu.gateway_request_id if gu else "",
provider           = gu.provider if gu else "",
provider_request_id = gu.provider_request_id if gu else None,
cache_hit          = gu.cache_hit if gu else False,
billing_units      = gu.billing_units if gu else 0,
cost_usd           = gu.cost_usd if gu else 0.0,
))

```

Key rule, carried from the HLD: local evidence first, external evidence only when policy-triggered, mutation only after mode and fitness gates pass. ToolRegistry.authorize_and_record_call enforces mode, policy trigger, and per-run call-count ceiling atomically.

Evidence Ledger vs. cost ledger (reconciliation). The two ledgers are kept separate and joined on gateway_request_id:

Record	Role	Holds
EvidenceLedger (\$9E)	Evidence / audit trail	Which claim, which source, under which policy signal & reason code
GatewayUsage	Per-call wire envelope	Cost subset carried into EvidenceLedgerEntry for audit
ProviderUsage (\$10A)	Normalized cost ledger	Full per-call cost record (native unit + optimus_credits_debited)

Reconciliation: EvidenceLedger.total_cost_usd() and the sum of ProviderUsage.cost_usd (joined by gateway_request_id) must equal the same figure.

10. Usage Accounting Service & TimeSeries Policy Enforcement

```
import redis.asyncio as aioredis
class UsageAccountingService:
    def __init__(
        self, pricing_snapshot_path: str,
        shared_redis_pool: aioredis.ConnectionPool,
    ):
        self.pricing_path = pricing_snapshot_path
        self.pool = shared_redis_pool
    async def _ensure_series_exists(
        self, client: aioredis.Redis, key: str, retention_ms: int,
    ):
        try:
            await client.execute_command(
                "TS.CREATE", key, "RETENTION", retention_ms
            )
        except aioredis.ResponseError as err:
            if "already exists" in str(err):
                await client.execute_command(
                    "TS.ALTER", key, "RETENTION", retention_ms
                )
            else:
                raise
```

```
def calculate_request_cost(
    self, usage_object: dict, model_alias: str, is_cached: bool,
) -> Tuple[float, bool, int]:
    input_tokens = usage_object.get("input_tokens", 0)
    output_tokens = usage_object.get("output_tokens", 0)
    is_fallback_pricing = False
    snapshot_timestamp = 0
    default_rates = {
        "input_rate": 1.40, "cached_rate": 0.26, "output_rate": 15.0,
    }
    if os.path.exists(self.pricing_path):
        with open(self.pricing_path, "r") as f:
            pricing_table = json.load(f)
            snapshot_timestamp = pricing_table.get(
                "meta", {}
            ).get("timestamp", 1774732000)
            model_rates = pricing_table.get(model_alias, None)
            if not model_rates:
                is_fallback_pricing = True
                model_rates = default_rates
    else:
        is_fallback_pricing = True
        model_rates = default_rates
    input_rate = (
        model_rates.get("cached_rate")
        if is_cached else model_rates.get("input_rate")
    )
    output_rate = model_rates.get("output_rate", 15.0)
```

```

cost = (
    (input_tokens * input_rate) + (output_tokens * output_rate)
) / 1000000
return cost, is_fallback_pricing, snapshot_timestamp

async def persist_telemetry_metrics(
    self, state_metadata: dict, cost: float,
    inputs: int, outputs: int,
):
    redis_client = aioredis.Redis(connection_pool=self.pool)
    ret_ms = 2592000000
    run_id = state_metadata.get("run_id")
    if not run_id:
        raise ValueError(
            "Access constraint violation: Missing telemetry run_id key."
        )
    c_key = f"telemetry:run:{run_id}:metrics:cost_usd"
    i_key = f"telemetry:run:{run_id}:metrics:tokens_input"
    o_key = f"telemetry:run:{run_id}:metrics:tokens_output"
    await self._ensure_series_exists(redis_client, c_key, ret_ms)
    await self._ensure_series_exists(redis_client, i_key, ret_ms)
    await self._ensure_series_exists(redis_client, o_key, ret_ms)
    async with redis_client.pipeline(transaction=True) as pipe:
        await pipe.execute_command("TS.ADD", c_key, "*", cost)
        await pipe.execute_command("TS.ADD", i_key, "*", inputs)
        await pipe.execute_command("TS.ADD", o_key, "*", outputs)
        ledger_item = state_metadata.get("assumption_ledger", {})
        if isinstance(ledger_item, dict):
            assumptions_list = ledger_item.get("assumptions", [])
        else:
            assumptions_list = getattr(ledger_item, "assumptions", [])
        h_key = f"run:{run_id}:metadata"
        await pipe.hset(h_key, mapping={
            "execution_mode": state_metadata.get("execution_mode", "PLAN"),
            "generation_scope": state_metadata.get("generation_scope", "INLINE_SNIPPET"),
            "rigor_level": state_metadata.get("rigor_level", "LOW"),
            "user_approval_id": state_metadata.get("user_approval_id", "unauthorized_direct_run"),
            "assumption_count": str(len(assumptions_list)),
        })
    await pipe.expire(h_key, 2592000)
    await pipe.execute()

```

A. Decoupled Redis Data Structure & Observability Trackers

To insulate the RedisTimeSeries metadata indices from high-cardinality label degradation over time, the storage engine decouples data tracking patterns:

- **Partitioned TimeSeries Keys:** System metrics scale inside dedicated key structures segmented explicitly by execution identifier strings:
telemetry:run:{run_id}:metrics:{metric_name}.
- **TimeSeries Policy Lifetime Management:** Keys feature an explicit retention window matching standard 30-day corporate storage stability intervals. Companion metadata descriptions are written to detached local Redis HASH data keys under layout patterns structured as run:{run_id}:metadata, configured with a corresponding automated expire

lifecycle policy window.

10A. Provider Usage Ledger Schema & Observability Export

This subsection extends Section 10 (Usage Accounting Service) and the Evidence Ledger Schema (§9E).

ProviderUsage / GatewayUsage Consistency

ProviderUsage extends GatewayUsage; it does not replace it. GatewayUsage remains the wire-level envelope returned by the Gateway on every billable call (as defined in §9E and §6). ProviderUsage is the canonical *persisted ledger record* and is a strict superset: it carries the GatewayUsage fields verbatim and adds the normalization fields (service, native_unit, optimus_credits_debited, price_snapshot_id). Usage accounting stores both the provider-native unit and the normalized internal charge for every request, so any upstream cost (tokens, Tavily credits, trace events) reconciles to a single Optimus ledger. The Evidence Ledger (§9E) is the evidence / audit trail and is joined to this cost ledger on gateway_request_id (audit ↔ cost); it does not duplicate the normalized cost fields.

```
class ProviderUsage(BaseModel):
    # --- GatewayUsage envelope fields (carried verbatim, §9E) ---
    gateway_request_id: str
    provider: str                # tavily | openai | langsmith | glm | ...
    provider_request_id: str | None = None
    cache_hit: bool = False
    billing_units: int = 0
    cost_usd: Decimal = Decimal("0")
    # --- ProviderUsage normalization extensions ---
    service: str                 # web.search | responses | trace.export
    native_unit: str              # tokens | tavily_credits | trace_events
    optimus_credits_debited: Decimal = Decimal("0") # normalized charge
    price_snapshot_id: str
```

The Gateway enforces budget against the wallet before dispatching any billable request; local budget state remains informational. optimus_credits_debited is the normalized figure carried into the per-run telemetry and the RedisTimeSeries cost series.

LangSmith Trace Export Mechanics

The canonical Phase 1 path (see §0A): the local agent sends structured trace events to the Gateway at /v1/observability/traces, and the Gateway forwards them to LangSmith with a server-side service key. The LangSmith key is never present locally; if LangSmith is proxied, LANGSMITH_ENDPOINT is forced to the Optimus Gateway. Each trace carries:

```
run_id, request_id, execution_mode, generation_scope,
model/provider (selected by Gateway), cache_hit, cost_usd,
billing_units, policy_signal, tool_class,
validation_outcome, failure_classification (when applicable)
```

11. Sprint 1 Implementation Checklist

Gates are grouped by subsystem. All gates must pass before the sprint is signed off. The Release Gate at the end is the single go/no-go criterion.

Transport & Protocol

- ☐ Verify StreamReader byte-by-byte header parsing with varying case forms of Content-Length.
- ☐ Validate that thread pool fallback behavior remains stable under a simulation flooding StreamReader with 50 sequential rapid bursts of fragmented headers.
- ☐ Ensure Task Manager instantly rejects duplicate sequence request IDs with an app-specific -32001 error payload.
- ☐ Verify path canonicalization validation errors are surfaced cleanly under cross-drive paths to trigger fail-closed alerts.

Runtime & Mode Enforcement

- ☐ Confirm that the `assert_mutation_allowed` primitive throws a clean Code -32002 validation error whenever a file modification or shell execution operation runs while `execution_mode` is bound to PLAN.
- ☐ Verify that a simulated failure in an internal composite gate rejects cleanly without mutating the active codebase tree.
- ☐ Confirm `authorize_and_record_call()` enforces `max_calls_per_run` atomically: no code path may record a call without authorizing it, or authorize a call without recording it.

Gateway & Authentication

- ☐ Verify `OptimusGatewaySettings` injects a valid Optimus Authorization bearer header (sourced from `optimus_api_key` as `SecretStr`) on every gateway call — model completions (`/v1/responses`), web search, and web extract.
- ☐ Validate `gateway_url` allowlist: production URL accepted; arbitrary URL rejected; enterprise URL in `OPTIMUS_EXTRA_GATEWAY_ORIGINS` (dev/test) accepted; same URL rejected when `production_mode=True` unless registered in signed tenant profile; URL not in any trusted set rejected with clear error.
- ☐ Verify Section 9D gateway server-side revalidation: allowed domains, extract URL provenance, budgets, call caps, and tool policies each enforced independently of the local agent, confirmed via integration tests that send policy-violating requests directly to the gateway.
- ☐ Confirm `optimus_api_key` never appears in plaintext in logs, telemetry, or serialized run state — `SecretStr` masking holds under `repr()`, `str()`, and `model_dump()`.

Evidence & Tools

- ☐ Confirm `EvidenceRequest.query` is sent verbatim to the gateway web search endpoint as the query field, and that reason is never substituted in its place.
- ☐ Verify `EvidenceExtractRequest` rejects any URL outside the prior approved search-result set and enforces the `max_chars_per_source` and `max-URL-count` bounds.
- ☐ Validate that `gateway_request_id`, `provider`, `provider_request_id`, `cache_hit`, `billing_units`, and `cost_usd` from both Search and Extract gateway responses are parsed into `GatewayUsage` and propagated to `EvidenceLedgerEntry`.

Telemetry & Accounting

- ☐ Confirm RedisTimeSeries entries execute idempotent TS.CREATE configurations or trigger automatic fallback TS.ALTER updates to consistently align 30-day RETENTION thresholds across both brand new and pre-existing keys.
- ☐ Assert that the usage accounting engine gracefully handles a missing pricing_snapshot.json file by mapping default fallback pricing matrices and raising a telemetry audit signal.
- ☐ Verify that the sharded Redis storage engine pipes active execution_mode, generation_scope, rigor_level, and assumption ledger parameters directly into the run metadata hash map on every workflow completion.
- ☐ Confirm EvidenceLedger.total_cost_usd() and total_billing_units() reconcile against the gateway's own usage accounting. total_credits() retained for backward compat and must also reconcile.
- ☐ Confirm ProviderUsage ledger records persist normalized optimus_credits_debited alongside provider-native billing_units for LLM, Tavily, and LangSmith usage (see §10A), joined to EvidenceLedgerEntry on gateway_request_id.

Coverage & Observability

- ☐ Confirm aggregate Python production-code coverage $\geq 80\%$ in CI via coverage.py + pytest-cov; safety-critical modules trend higher and do not regress (authoritative detail: Test Strategy §8A).
- ☐ Confirm LangSmith trace export uses the canonical path (agent → Gateway → LangSmith) with server-side credentials; no LANGSMITH_API_KEY is resolvable locally.

Sprint 1 Release Gate

RELEASE GATE — One-key setup end to end: verify the local agent starts and completes a full Plan-mode and Agent-mode run with only OPTIMUS_GATEWAY_URL and OPTIMUS_API_KEY configured. Confirm no provider API key (Tavily, OpenAI, OpenRouter, GLM, **LangSmith**, or any other upstream credential) is resolvable from the local environment, config files, or process state at any point during the run. This gate must pass before the sprint is marked complete.

11A. Test Coverage & Observability — Cross-Reference

This is a short cross-reference under the test / observability / release-gate section. The authoritative detail lives in the Test Strategy (§8A); the HLD and LLD may summarize the target, but Test Strategy §8A remains authoritative.

Test coverage for production Python code is measured with **coverage.py** (canonical line and branch coverage) and **pytest-cov** (CI and local integration). The Phase 1 release gate is a **minimum 80% aggregate Python coverage** threshold; safety-critical modules (ToolRegistry authorization, MutationGuard, GatewayUsage / ProviderUsage accounting, EvidenceLedger reconciliation, JSON-RPC framing, retry / fail-closed logic, policy enforcement) trend materially higher and must not regress.

Agent-quality and AI-safety tools are tracked separately and do **not** count toward the coverage metric: **DeepEval** and **Ragas** for agent / evidence-quality evaluation, and **PyRIT** for adversarial / red-team probes. **LangSmith** traces are used for debugging and regression analysis, not as a coverage metric. Critical failures from the DeepEval / Ragas / PyRIT suites do not affect the coverage figure but may still block release through separate quality and security gates. Authoritative thresholds, CI commands, eval suites, red-team tests, LangSmith trace assertions, and release gates are defined in the Test Strategy document, §8A.

12. Guardrail & Workflow Component Contracts

This section specifies the Phase 1 enforcement and workflow components introduced as a cross-cutting concern in **HLD §13**. The authoritative policy rationale lives in the companion **Agent Execution Guardrails & Workflow Strategy v1.0** (§2-§9); the representative pydantic type shapes are in its §10.2. The contracts below are owned by this section. All model-touching elements (the borderline classifier and the loop completion evaluator) route through the Optimus Gateway under the existing budget wallet and normalized ledger (§0A, §10A); the guardrails introduce no second cost path.

12A. Permission & Pre-Tool Enforcement

- **PermissionPolicy** — evaluates `mode` → `user_deny` → `project_allow` → `impact` → `classifier` and returns a single `PermissionDecision` (`ALLOW` | `DENY` | `HOLD`); deny always precedes allow; the classifier cannot overturn a deny.
- **PermissionDecision** — `verdict`, `layer`, `rule_id`, `reason`, `requires_human_approval`.
- **PreToolGuard** — fires after the agent assembles a tool call, before execution (cf. `PreToolUse`), for every tool class (`bash`, `file_edit`, `mcp_call`, `web`); deterministic checks run before any classifier. Returns `PreToolResult`.
- **CommandSafetyValidator** — local, in-process shell validator covering destructive commands, pipe-to-shell, environment/credential access, Unicode homoglyph/confusable characters, ANSI/control sequences, insecure transport, and unexpected network egress. Satisfiable by **Tirth or an equivalent validator**; the design takes no hard dependency on any one implementation.
- **ToolInvocationAuditEvent** — append-only record of each decision (`verdict`, `layer`, failed checks, approver) feeding the same trace sink as evidence and cost telemetry.

12B. Prompt-Injection & MCP Supply-Chain Trust

- **MCPTrustRegistry** — captures `server_id`, `manifest_hash`, `allowed_tools`, `permission_scope`, and `approved`. MCP servers are never auto-loaded from cloned repositories; a manifest-hash change forces re-approval; tool descriptors (`name/description/schema`) are inspected for injection before being surfaced to the planner.
- **ConfigTrustScanner** — treats agent config and rule files as code, scanning on ingest for embedded instructions, exfiltration endpoints, and homoglyph/ANSI content before they may influence behavior.

12C. Bounded Agent Loops

- **GoalLoopController**, `IterationState`, `CompletionEvaluator`, `ProgressLedger`, `LoopBudgetPolicy` (`max_iterations`, `max_budget_credits`, `max_wall_clock_minutes`), and `LoopStopReason` (`COMPLETED` | `MAX_ITERATIONS` | `BUDGET_EXHAUSTED` | `WALL_CLOCK` | `REPEATED_FAILURE` | `HUMAN_HALT`).
- Persistent state lives in files, git history, task manifests, traces, and the evidence ledger (§9E), not in an ever-growing chat context. The pre-tool guard (§12A) is never bypassed inside a loop, and the completion evaluator is a cheap Gateway-routed model, not the main reasoning model.

12D. Curated Workflow Skills

- SkillRegistry, SkillManifest (name, description, globs, allowed_tools, owner, version, trust_level), SkillTrustPolicy (draft/untrusted skills blocked in Agent mode), and SkillInvocationPolicy.
- A skill's declared allowed_tools are enforced by the pre-tool guard (§12A) — a skill can never widen the agent's tool surface — and a skill can never override project or user deny rules.